OrderResource.php Explained

This handout explains how `OrderResource.php` works in a Laravel + Filament project. The file is responsible for building the order form, computing totals, validating payment, and showing saved orders in the table view.

Laravel | Filament Resource | Order Form | Student Notes

1. File Header and Imports

This first part tells PHP where the class belongs and which classes it needs to use. The `use` statements import Filament form components, table components, models, and helper classes.

```
<?php

namespace App\Filament\Resources;

use App\Filament\Resources\OrderResource\Pages;
use App\Filament\Resources\OrderResource\RelationManagers;
use App\Models\Order;
use App\Models\Product;
use Closure;
use Filament\Forms;
use Filament\Forms\Components\Repeater;
use Filament\Forms\Components\Section;
use Filament\Forms\Components\Select;
use Filament\Forms\Components\TextInput;
use Filament\Forms\Form;
use Filament\Forms\Get;
use Filament\Forms\Set;
use Filament\Resources\Resource;
use Filament\Tables;
use Filament\Tables\Columns\TextColumn;
use Filament\Tables\Table;
use Illuminate\Database\Eloquent\Builder;
```

```
use Illuminate\Database\Eloquent\SoftDeletingScope;
use Illuminate\Validation\ValidationException;
```

> Important idea: importing classes with `use` makes the file shorter and easier to read.

2. Class Declaration

The class extends Filament's `Resource`. That means this file defines a CRUD interface for the `Order` model inside the admin panel.

```
class OrderResource extends Resource
{
    protected static ?string $model = Order::class;

    protected static ?string $navigationIcon = 'heroicon-o-rectangle-
stack';
```

- `$model` connects the resource to the `Order` model.
- `$navigationIcon` chooses the sidebar icon in Filament.

3. The form() Method

The `form()` method builds the Create and Edit form for orders. Inside this method, there are helper functions used to calculate totals and update values automatically.

```
public static function form(Form $form): Form
{
    $calculations = function (Get $get, Set $set) {
        $items = $get('../../orderItems');
        $total = 0;
```

```
        if (is_array($items)) {
            foreach ($items as $item) {
                $product = Product::find($item['product_id'] ??
null);
                if ($product) {
                    $total += $product->cost_price * (float)
($item['quantity'] ?? 0);
                }
            }
        }

        $set('total', $total);
    };
```

- `$calculations` loops through all order items.
- For each item, it gets the selected product from the database.
- It multiplies `cost_price × quantity`.
- It adds all subtotals together and stores the result in the `total` field.

> This closure is used when the product or quantity changes so the total updates live.

4. Duplicate Total Helper

This second helper copies the value of `total` into another field named `output_total_duplicate`.

```
$totalValue = function (Get $get, Set $set) {
    $getValue = $get('total');

    $set('output_total_duplicate', $getValue);
};
```

In simple terms, this is a value-mirroring function. When triggered, it reads the current total and writes that same number to another input.

5. Change Calculation Helper

This function appears to calculate change, but in the current file it is never actually called. That means it does not affect the form unless you connect it somewhere.

```
$changeFn = function (Get $get, Set $set) {
    $order = $get('../../Order');
    $amountChange = 0;

    if ($order) {
        $ordered_get = Order::find($order['user_id'] ?? null);

        if ($ordered_get) {
            $amountChange = $ordered_get->amountTendered -
$ordered_get->total;
        }

        $set('change', $amountChange);
    }
};
```

> Teaching note: this helper looks unfinished or unused. The actual change calculation is done later inside the `amount_tendered` field.

6. Customer Section

The form starts by asking which user placed the order.

```
return $form
    ->schema([
        Section::make('Customer\'s Detail')
            ->schema([
                Select::make('user_id')
                    ->relationship('user', 'name')
```

```
                    ->searchable(),
            ])
            ->compact(),
```

- `Select::make('user_id')` creates a dropdown for the user.
- `relationship('user', 'name')` tells Filament to use the related `user` model and show the `name`.
- `searchable()` lets the admin search users quickly.

7. Ordering Section and Repeater

The repeater allows the admin to add many ordered products in one order. Each repeated block represents one row in the order items list.

```
Section::make('Ordering Section')
    ->schema([
        Repeater::make('orderItems')
            ->columnSpanFull()
            ->relationship()
            ->schema([
                Select::make('product_id')
                    ->relationship('product', 'name')
                    ->live(debounce: 1000)
                    ->afterStateUpdated(function (Set $set, $state) {
                        $product = Product::find($state);
                        $set('purchase_price', $product->cost_price
?? 0);
                    }, $calculations)
                    ->required()
                    ->validationMessages([
                        'required' => 'Please select a product'
                    ])
                    ->searchable()
                    ->columnSpanFull(),
```

- `Repeater::make('orderItems')` is used because an order can contain many products.
- The repeater uses the model relationship, so each row is saved as an order item.
- When a product is selected, the code finds that product and copies its `cost_price` into `purchase_price`.
- `live(debounce: 1000)` means it waits 1 second before reacting, which reduces excessive updates.

8. Purchase Price and Quantity Fields

```
TextInput::make('purchase_price')
    ->disabled()
    ->dehydrated()
    ->numeric(),

TextInput::make('quantity')
    ->numeric()
    ->live(debounce: 1000)
    ->disabled(fn(Get $get) => $get('product_id') === null)
    ->afterStateUpdated($calculations)
    ->required(),
```

- `purchase_price` is disabled so the user cannot manually change it.
- `dehydrated()` means the disabled field is still included when the form is saved.
- `quantity` stays disabled until a product is selected.
- When quantity changes, the total is recalculated.

> This is a good example of dependent form fields: one field becomes available only after another field is filled in.

9. Total and Payment Fields

```
TextInput::make('total')
    ->numeric()
    ->disabled()
    ->dehydrated(),

TextInput::make('amount_tendered')
    ->numeric()
    ->live(debounce: 1000)
    ->disabled(fn(Get $get) => $get('total') === null || (float)
$get('total') === 0.0)
    ->rules([
        fn(Get $get): Closure => function (string $attribute, $value,
Closure $fail) use ($get) {
            $total = (float) $get('total');
            $amountTendered = (float) $value;
            if ($total > $amountTendered) {
                $fail('Amount tendered should be greater than or
equal to total.');
            }
        },
    ])
    ->afterStateUpdated(function (Set $set, Get $get, $state) {
        $change = (float) $get('amount_tendered') - (float)
$get('total');
        $set('change', $change);
    })
    ->required(),
```

- `total` is auto-computed and cannot be typed manually.
- `amount_tendered` is disabled until the total is greater than zero.
- A custom validation rule checks that the customer pays enough money.
- After updating the amount tendered, the code computes `change = amount_tendered - total`.

10. Duplicate Total Inputs and Change Output

```
TextInput::make('input_total_duplicate')
    ->live(debounce: 1000)
    ->afterStateUpdated($totalValue),
TextInput::make('output_total_duplicate'),
TextInput::make('change')
    ->numeric()
    ->disabled()
```

- `input_total_duplicate` is a trigger field that runs the duplicate-total helper.
- `output_total_duplicate` receives the copied total value.
- `change` shows the remaining money after payment.

> For classroom discussion: the duplicate-total fields are unusual in a real order form. They may have been added for practice, testing, or experimentation.

11. Table Display

The `table()` method controls how saved orders appear in the Filament list page.

```
public static function table(Table $table): Table
{
    return $table
        ->columns([
            TextColumn::make('id'),
            TextColumn::make('orderItems.product.name')
                ->listWithLineBreaks(),
            TextColumn::make('orderItems.purchase_price')
                ->listWithLineBreaks(),
            TextColumn::make('orderItems.quantity')
                ->listWithLineBreaks(),
            TextColumn::make('sub_total')
                ->state(fn($record) => $record->orderItems->map(
                    fn($item) => $item->purchase_price * $item-
>quantity
```

```
                ))
                ->listWithLineBreaks()
                ->money('PHP')
                ->label('Sub Total'),
            TextColumn::make('total')
                ->money('PHP'),
            TextColumn::make('created_at')
                ->date('m/d/y')
        ])
```

- `TextColumn::make('orderItems.product.name')` shows product names from related order items.
- `listWithLineBreaks()` displays multiple values on separate lines.
- The `sub_total` column is custom and calculates each item's subtotal using `purchase_price × quantity`.
- `money('PHP')` formats values as Philippine peso currency.

12. Actions and Bulk Actions

```
->filters([
    //
])
->actions([
    Tables\Actions\EditAction::make(),
    Tables\Actions\DeleteAction::make(),
])
->bulkActions([
    Tables\Actions\BulkActionGroup::make([
        Tables\Actions\DeleteBulkAction::make(),
    ]),
]);
```

- No table filters are defined yet.
- The resource supports editing and deleting one order at a time.

- It also supports deleting multiple selected orders at once.

13. Relations and Pages

```
public static function getRelations(): array
{
    return [
        //
    ];
}

public static function getPages(): array
{
    return [
        'index' => Pages\ListOrders::route('/'),
        'create' => Pages\CreateOrder::route('/create'),
        'edit' => Pages\EditOrder::route('/{record}/edit'),
    ];
}
```

- `getRelations()` is empty, so no extra relation managers are attached.
- `getPages()` defines the three main routes: list, create, and edit.

14. Summary for Students

What this file does

- Builds the order input form
- Lets the user select customer and products
- Computes total automatically
- Validates payment amount
- Calculates customer change
- Displays saved orders in a table

Important concepts used

- Filament Resource
- Form schema
- Repeater fields
- Relationships
- Live field updates
- Custom validation rules

15. Full Original Code

This is the full original `OrderResource.php` source included for reference.

```
<?php

namespace App\Filament\Resources;

use App\Filament\Resources\OrderResource\Pages;
use App\Filament\Resources\OrderResource\RelationManagers;
use App\Models\Order;
use App\Models\Product;
use Closure;
use Filament\Forms;
use Filament\Forms\Components\Repeater;
use Filament\Forms\Components\Section;
use Filament\Forms\Components\Select;
use Filament\Forms\Components\TextInput;
use Filament\Forms\Form;
use Filament\Forms\Get; //get value inside the textinput
use Filament\Forms\Set; //set new value inside the textinput
use Filament\Resources\Resource;
use Filament\Tables;
use Filament\Tables\Columns\TextColumn;
use Filament\Tables\Table;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\SoftDeletingScope;
```

```
use Illuminate\Validation\ValidationException;

class OrderResource extends Resource
{
    protected static ?string $model = Order::class;

    protected static ?string $navigationIcon = 'heroicon-o-rectangle-
stack';

    public static function form(Form $form): Form
    {
        $calculations = function (Get $get, Set $set) {
            $items = $get('../../orderItems');
            $total = 0;

            if (is_array($items)) {
                foreach ($items as $item) {
                    $product = Product::find($item['product_id'] ??
null);
                    if ($product) {
                        $total += $product->cost_price * (float)
($item['quantity'] ?? 0);
                    }
                }
            }

            $set('total', $total);
        };

        $totalValue = function (Get $get, Set $set) {
            $getValue = $get('total');

            $set('output_total_duplicate', $getValue);
        };

        $changeFn = function (Get $get, Set $set) {
            $order = $get('../../Order');
            $amountChange = 0;

            if ($order) {
                $ordered_get = Order::find($order['user_id'] ??
null);
```

```
                if ($ordered_get) {
                    $amountChange = $ordered_get->amountTendered -
$ordered_get->total;
                }

                $set('change', $amountChange);
            }
        };

        return $form
            ->schema([
                Section::make('Customer\'s Detail')
                    ->schema([
                        Select::make('user_id')
                            ->relationship('user', 'name')
                            ->searchable(),
                    ])
                    ->compact(),
                Section::make('Ordering Section')
                    ->schema([
                        Repeater::make('orderItems')
                            ->columnSpanFull()
                            ->relationship()
                            ->schema([
                                Select::make('product_id')
                                    ->relationship('product', 'name')
                                    ->live(debounce: 1000)
                                    ->afterStateUpdated(function (Set
$set, $state) {
                                        $product =
Product::find($state); //query
                                        $set('purchase_price',
$product->cost_price ?? 0);
                                    }, $calculations)
                                    ->required()
                                    ->validationMessages([
                                        'required' => 'Please select
a product'
                                    ])
                                    ->searchable()
                                    ->columnSpanFull(),
                                TextInput::make('purchase_price')
                                    ->disabled()
```

```
                                    ->dehydrated()
                                    ->numeric(),


                                TextInput::make('quantity')
                                    ->numeric()
                                    ->live(debounce: 1000)
                                    ->disabled(fn(Get $get) =>
$get('product_id') === null)
                                    -
>afterStateUpdated($calculations)
                                    // ->afterStateUpdated(function
(Get $get, Set $set, $component) use ($calculations) {
                                    //     if ($get('product_id') ==
null) {
                                    //         throw
ValidationException::withMessages([
                                    //             $component-
>getStatePath() => 'Please select product first.',
                                    //         ]);
                                    //     }
                                    //     $calculations($get, $set);
                                    // })
                                    ->required(),
                            ]),
                        TextInput::make('total')
                            ->numeric()
                            ->disabled()
                            ->dehydrated(),

                        TextInput::make('amount_tendered')
                            ->numeric()
                            ->live(debounce: 1000)
                            ->disabled(fn(Get $get) => $get('total')
=== null || (float) $get('total') === 0.0)
                            ->rules([
                                fn(Get $get): Closure => function
(string $attribute, $value, Closure $fail) use ($get) {
                                    $total = (float) $get('total');
                                    $amountTendered = (float) $value;
                                    if ($total > $amountTendered) {
                                        $fail('Amount tendered should
be greater than or equal to total.');
```

```
                                    }
                                },
                            ])
                            ->afterStateUpdated(function (Set $set,
Get $get, $state) {
                                $change = (float)
$get('amount_tendered') - (float) $get('total');
                                $set('change', $change);
                            })
                            ->required(),
                        TextInput::make('input_total_duplicate')
                            ->live(debounce: 1000)
                            ->afterStateUpdated($totalValue),
                        TextInput::make('output_total_duplicate'),
                        TextInput::make('change')
                            ->numeric()
                            ->disabled()


                    ])
                    ->columns(2)
                    ->compact(),
            ]);
    }

    public static function table(Table $table): Table
    {
        return $table
            ->columns([
                TextColumn::make('id'),
                TextColumn::make('orderItems.product.name')
                    ->listWithLineBreaks(),
                TextColumn::make('orderItems.purchase_price')
                    ->listWithLineBreaks(),
                TextColumn::make('orderItems.quantity')
                    ->listWithLineBreaks(),
                TextColumn::make('sub_total')
                    ->state(fn($record) => $record->orderItems->map(
                        fn($item) => $item->purchase_price * $item-
>quantity
                    ))
                    ->listWithLineBreaks()
                    ->money('PHP')
```

```
                    ->label('Sub Total'),
                TextColumn::make('total')
                    ->money('PHP'),
                TextColumn::make('created_at')
                    ->date('m/d/y')

            ])
            ->filters([
                //
            ])
            ->actions([
                Tables\Actions\EditAction::make(),
                Tables\Actions\DeleteAction::make(),
            ])
            ->bulkActions([
                Tables\Actions\BulkActionGroup::make([
                    Tables\Actions\DeleteBulkAction::make(),
                ]),
            ]);
    }

    public static function getRelations(): array
    {
        return [
            //
        ];
    }

    public static function getPages(): array
    {
        return [
            'index' => Pages\ListOrders::route('/'),
            'create' => Pages\CreateOrder::route('/create'),
            'edit' => Pages\EditOrder::route('/{record}/edit'),
        ];
    }
}
```